

UIPilot User Guide

Setup, the one-click workflow, everything UIPilot creates, how to make it yours, and UI Health, which checks all your UI as you work.

Open the tool: [Tools > UIPilot](#)

UIPilot is a Unity Editor tool. It builds a Main Menu, Pause Menu and Settings Menu out of standard Unity UI (UGUI + TextMeshPro), writes a small script for the button logic, connects every button to that script, and can check and repair the result later.

It also watches **all** the UI in your scenes as you work, not only its own. **UI Health** (section 6) catches the mistakes that make a button silently do nothing (a click event pointing at a deleted method, a missing raycaster, an invisible image covering the button, a control a gamepad can never reach) and text too small for Steam Deck, and fixes most of them in one click. In Play mode, the **Click Debugger** (section 7) tells you what took each click and why, so "this button does nothing" takes one click to explain.

Everything it creates is ordinary Unity UI that you edit in the Inspector. UIPilot's own code runs only in the Editor. The one runtime file is `UIPilot_GameManager.cs`, which UIPilot writes into your project for you to own and edit.

1. Requirements

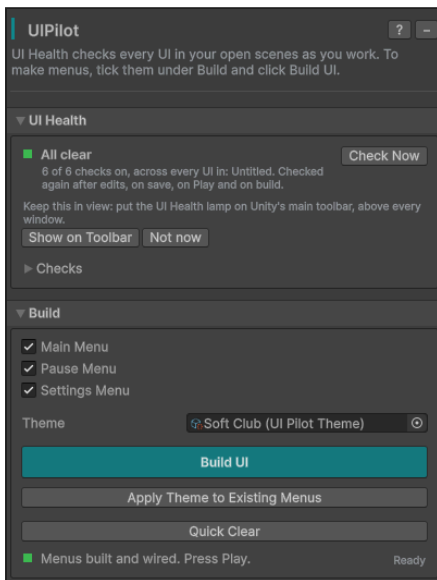
Unity	6.3 LTS (6000.3) or newer. Tested on 6000.3.11f1, Windows.
Packages	Unity UI (<code>com.unity.ugui</code>), which includes TextMeshPro. Installed by default in every Unity 6 project.
TextMeshPro resources	TMP Essential Resources must be imported (step 2 below).
Input	Works with every <i>Active Input Handling</i> setting: Input Manager (Old), Input System Package (New), or Both. UIPilot picks the matching input module for you.
Render pipeline	Tested with the Built-in Render Pipeline. The menus use a Screen Space - Overlay canvas, which does not depend on the render pipeline.

2. Setup (about one minute)

1. **Import UIPilot.** Everything is placed in `Assets/UIPilot/`.

2. **Import the TextMeshPro essentials**, if your project does not have them yet: **Window > TextMeshPro > Import TMP Essential Resources**, then click **Import**. You can skip this if your project already contains an `Assets/TextMesh Pro` folder. Without these resources, menu text cannot be displayed.
3. **Open the scene** you want the menus in. Any scene works, including an empty one.
4. **Open UIPilot: Tools > UIPilot**. Dock the window wherever you like.

3. Quick start: build your menus



What Build UI creates: the main menu, rendered in Unity 6000.3.11f1. The title and version are this project's own (its Product Name is "UIPilot"). The first button has focus, the second is under the mouse. The shapes behind are a stand-in scene, seen through the wash.

The UIPilot window. UI Health (section 6) is on top. Under Build, tick the menus, click **Build UI**, and the row beneath the buttons reports the result.

1. In the **Build** section, tick the menus you want. All three are ticked by default: *Main Menu*, *Pause Menu*, *Settings Menu*.
2. Click **Build UI**.
3. **The first time only, wait a few seconds.** UIPilot has just written a new script, and Unity needs to compile it. The Console shows:

```
UIPilot: Waiting for Unity to compile UIPilot_GameManager.cs — buttons will be wired automatically when it finishes.
```

You do not need to click anything. When Unity finishes compiling, the Console shows:

```
UIPilot: Setup complete. Your buttons are wired and ready.
```

The UIPilot window shows the same progress under the Build UI button: an amber *Working* row while Unity compiles, then a green *Ready* row.

4. **Press Play.** The main menu appears, showing your project's name and version, with the first button already focused.
 - Use the **mouse**, or the **arrow keys / gamepad stick** to move and **Enter / gamepad A** to press. No extra setup is needed for keyboard or gamepad.

- **Settings** opens a Settings menu that already works: **Volume**, **Fullscreen** and **Quality** change the real settings and are remembered the next time the game runs. **Back** returns to where you came from.
- **Quit** stops Play mode in the Editor, and quits the application in a build.
- **Play** writes a message to the Console. It does nothing else until you add your own code (see section 5.1).

That is the whole workflow. The rest of this guide explains what was created and how to make it yours.

4. What UIPilot creates



Pause menu



Settings menu. All three rows work: here the volume has been turned down to 70%.

In the Hierarchy

UIPilot_Canvas	Canvas (Screen Space - Overlay), GraphicRaycaster, CanvasScaler: Scale With Screen Size, 1920 x 1080, Match 0.5
UIPilot_MainMenu_Panel	full-screen colour wash over the scene + layout group
UIPilot_MainMenu_Backdrop	soft colour blooms, thin frames, the ribbon of light
UIPilot_MainMenu_Band	the glass panel: scanlines, hairline frame and its echo
UIPilot_MainMenu_Title	your Product Name (shown in lowercase)
UIPilot_MainMenu_Rule	hairline under the title
UIPilot_MainMenu_Spacer	pushes the buttons to the bottom of the glass
UIPilot_Btn_Play	each button has three children: Bar, Icon and Text
UIPilot_Btn_Settings	
UIPilot_Btn_Quit	
UIPilot_MainMenu_Footnote	small print: your project's version
UIPilot_PauseMenu_Panel	same structure: Resume, Settings, Quit
UIPilot_Settings_Panel	Band, Title, Rule, then:
UIPilot_Settings_Row_Volume	Label, "<" button, value + meter, ">" button
UIPilot_Btn_Fullscreen	a button that shows On / Off on its right
UIPilot_Settings_Row_Quality	Label, "<" button, value, ">" button
UIPilot_Settings_Spacer	
UIPilot_Btn_Back	
EventSystem	created only if the scene does not have one
UIPilot_GameManager	empty GameObject holding the UIPilot_GameManager component

Every button's **On Click ()** list in the Inspector points at the matching method on `UIPilot_GameManager`: `UIPilot_Btn_Play` calls `OnPlayPressed`, and so on.

In the Project window

File	Purpose
<code>Assets/UIPilot_GameManager.cs</code>	Your script. One method per button, plus the code that shows and hides the panels. Edit it freely.
<code>Assets/UIPilot/Themes/</code>	The looks you can choose from: Soft Club, Soft Club Night, Ink, Fantasy RPG, JRPG Window, Pixel Retro and Sci-Fi HUD. Duplicate one to make your own (section 5.4). Editor-only; not part of your build.
<code>Assets/UIPilot/Art/</code>	The menus' artwork: sprites, icons and fonts. The generated menus use these files, so keep this folder in your project (you may move it). The Soft Club sprites are white, so you recolour them in the Inspector instead of repainting. The genre themes' artwork is in <code>Art/Themes</code> and carries its own colours.

In Edit mode, all panels are visible at once

The panels are full-screen and stacked, so in the Scene and Game views you see the last one (Settings) on top. This is normal. When you press Play, the script shows only the main menu.

To work on one panel, untick the others in the Inspector. UIPilot still finds unticked panels, and the script turns the right panel on when the game runs.

5. Making it yours

5.1 Add your game logic

Open `Assets/UIPilot_GameManager.cs`. Each button has a method. Replace the placeholder in `OnPlayPressed` with your own code, for example:

```
using UnityEngine.SceneManagement; // add at the top of the file

public void OnPlayPressed()
{
    SceneManager.LoadScene("GameScene");
}
```

If the game runs in the same scene as the menu, hide the menu instead:

```
public void OnPlayPressed()
{
    ShowPanel(string.Empty); // an empty name hides every panel
}
```

5.2 Open the pause menu

UIPilot builds the pause menu and wires its buttons, but it does not decide which key pauses your game. Add this small script to any `GameObject` in the scene:

```

using UnityEngine;

public class PauseInput : MonoBehaviour
{
    [SerializeField] private UIPilot_GameManager gameManager;

    private void Update()
    {
        if (!PausePressed()) return;

        Time.timeScale = 0f;
        gameManager.ShowPanel("UIPilot_PauseMenu_Panel");
    }

    private static bool PausePressed()
    {
#if ENABLE_INPUT_SYSTEM
        var keyboard = UnityEngine.InputSystem.Keyboard.current;
        return keyboard != null && keyboard.escapeKey.wasPressedThisFrame;
#else
        return Input.GetKeyDown(KeyCode.Escape);
#endif
    }
}

```

Drag the `UIPilot_GameManager` object onto the **Game Manager** field. The generated **Resume** button already sets `Time.timeScale` back to 1 and hides the menu.

5.3 The Settings menu

The three rows work as soon as the menu is built. Nothing needs wiring or coding.

Row	What it changes	Buttons and the methods they call
Volume	<code>AudioListener.volume</code> , in steps of 10%. The meter under the number follows it.	<code>UIPilot_Btn_VolumeDown</code> → <code>OnVolumeDownPressed()</code> , <code>UIPilot_Btn_VolumeUp</code> → <code>OnVolumeUpPressed()</code>
Fullscreen	<code>Screen.fullScreen</code> . Press the row to switch between On and Off. Unity ignores this inside the Editor's Game view, so check it in a build.	<code>UIPilot_Btn_Fullscreen</code> → <code>OnFullscreenPressed()</code>
Quality	<code>QualitySettings.SetQualityLevel</code> . It steps through the levels in your project's Quality settings and shows each level's real name.	<code>UIPilot_Btn_QualityDown</code> → <code>OnQualityDownPressed()</code> , <code>UIPilot_Btn_QualityUp</code> → <code>OnQualityUpPressed()</code>

The player's choices are saved with `PlayerPrefs` (keys `UIPilot.Volume`, `UIPilot.Fullscreen`, `UIPilot.Quality`) and applied again the next time the game starts. All of this lives in `ApplySettings()` in `UIPilot_GameManager.cs`, which is yours to change.

To add a setting of your own:

1. In the Hierarchy, duplicate `UIPilot_Settings_Row_Quality` and move the copy where you want it.

2. Rename the copy's two buttons, for example `UIPilot_Btn_BrightnessDown` and `UIPilot_Btn_BrightnessUp`, and change its `Label` text. The name after `UIPilot_Btn_` decides which method the button calls.
3. In `UIPilot_GameManager.cs`, add `public void OnBrightnessDownPressed()` and `public void OnBrightnessUpPressed()`, and set the row's value text from them.
4. Back in UIPilot, click **Build UI**. Nothing is rebuilt, but every button is reconnected to the method that matches its name, so the two new buttons now call your two new methods instead of the Quality ones they were copied from.

5.4 Themes



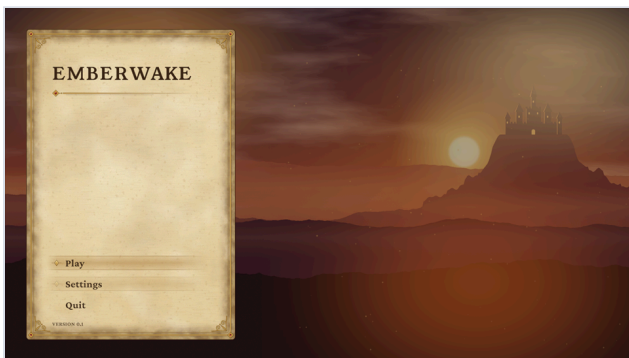
Soft Club (default)



Soft Club Night



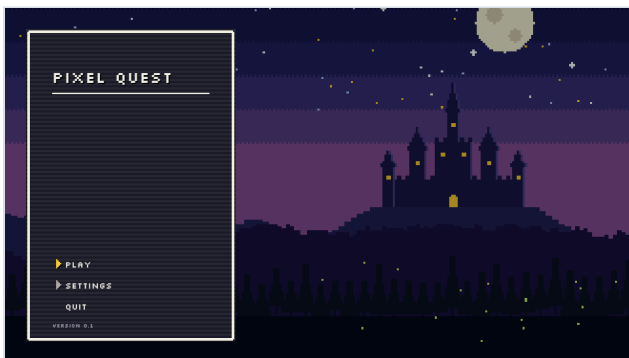
Ink



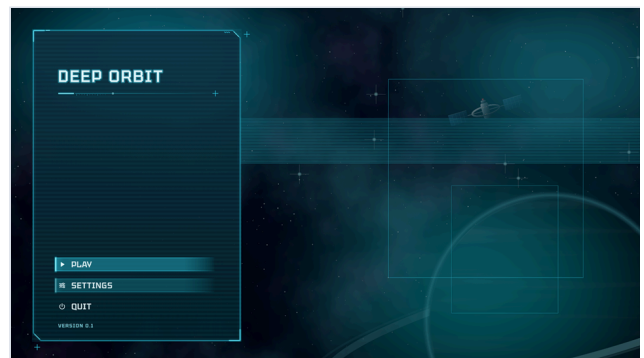
Fantasy RPG



JRPG Window



Pixel Retro



Sci-Fi HUD. Each genre theme is shown over a scene drawn for it; the game names are made up.

A **theme** is one asset that decides how every generated menu looks: the colours, the lettering, the artwork, and which decorations are switched on. UIPilot ships seven, in `Assets/UIPilot/Themes`:

Theme	The look
Soft Club (default)	Airy blue wash over your scene, cobalt glass panel, hairline frames, a ribbon of light, wide lowercase lettering.

Theme	The look
Soft Club Night	The same artwork in a dark key: deep navy, violet and cyan light.
Ink	Dark, flat and quiet: an opaque panel, a teal focus bar, the default font in capitals, no artwork. High contrast and genre-neutral.
Fantasy RPG	A parchment page in a gilt frame with garnet-set corners, lit by candlelight. Serif lettering in ink brown, a gold-leaf band with a lozenge on the chosen line, a gilt divider under the title.
JRPG Window	The console-RPG window: a blue gradient box with a white double border, bold capitals with a drop shadow, a pointing glove as the cursor.
Pixel Retro	8-bit: a black window with a white stepped border drawn in true pixels, pixel capitals, a yellow arrow cursor, faint CRT lines. The window and cursor are drawn at exactly 4 screen pixels per art pixel at 1080p.
Sci-Fi HUD	A starship console: a glass slab with cut corners edged in glowing cyan, instrument ticks, a scanning focus bar, technical capitals with a faint glow.

To choose a theme: in the **Build** section, drop it into the **Theme** field (or click the circle beside the field and pick one), then click **Build UI**.

To change the theme of menus you have already built: pick the theme, then click **Apply Theme to Existing Menus**. Only the appearance changes. Your layout, object names, button order and button wiring are left exactly as they are, and one **Ctrl+Z** undoes it. Apply overwrites colours and fonts you changed by hand on those menus, so if you like tweaking by hand, tweak the theme instead.

To make your own theme: select a theme in the Project window, press **Ctrl+D** to duplicate it, rename the copy, and edit it in the Inspector. Every field has a tooltip. Pick your copy in the **Theme** field and click **Apply Theme to Existing Menus** to see it. (Right-click in the Project window and choose **Create > UIPilot > Theme** to make a new one. It starts as a full copy of Soft Club, font and title glow included.)

To get a default theme back: if one of the seven is missing from the project, a line under the **Theme** field says which, next to a **Restore Default Themes** button. Click it and the missing themes are recreated in `Assets/UIPilot/Themes`, exactly as they shipped. It never changes a theme that is still there, so to reset a default theme you have edited, delete it first, then click **Restore Default Themes**.

Things worth knowing when you edit a theme:

- **Panel** is the surface the menu text is read on. Keep its alpha near 1, and check **Text** against it: aim for a contrast of 4.5:1 or better.
- **Focus Style** decides how the chosen button is marked: **Frame** (a glowing frame around it), **Bar** (a band behind it) or **Marker** (a cursor before its words, like the JRPG glove). **Focus Art** is the sprite for it; empty gives UIPilot's own frame, or a flat bar. **Focus Selected** tints it on the focused button, **Focus Hover** under the mouse. Keep **Focus Idle** at alpha 0. A theme whose art carries its own colours uses white for Selected, so the art shows as drawn.
- **Panel Art** is an optional 9-sliced sprite drawn as the panel (the parchment, the JRPG window). **Pixel Size** scales its border: the Pixel Retro window uses 0.25, so each art pixel covers four screen pixels.
- **Rule Art** replaces the thin line under the title with an ornament, at **Rule Height**.

- **Font** empty = TextMeshPro's default font. **Title Font** gives titles their own font (Fantasy RPG pairs a display serif with a book face). **Title Material** and **Text Material** are optional material presets *of the font they go with*: a glow, a drop shadow.
- **Icons** off hides the icons before item labels. Markers need that space.
- A theme never adds or removes objects. Every menu contains every decoration; a theme only switches them on and off. That is why any theme can be applied over any other.
- Themes are Editor-only assets. They are not included in your build. You can share one with a teammate or another project by copying the `.asset` file.

5.5 Change the look by hand

Everything is a standard component, so use the Inspector as usual. This is the way to change one menu or one button; to change every menu at once, edit the theme instead (section 5.4). The table describes the default Soft Club theme. Remember that **Apply Theme to Existing Menus** sets colours and fonts again, so it replaces these hand edits.

To change	Select	Edit
The game title	UIPilot_MainMenu_Title	It is filled in from Edit > Project Settings > Player > Product Name when the menu is built. To change it afterwards, edit TextMeshPro > Text. Type the name normally: it is shown in lowercase by the Lowercase font style (the ab button), which you can switch off. A long name shrinks, then wraps, to stay inside the menu column.
The version line	UIPilot_MainMenu_Footnote	Nothing to edit: the script sets it to Player > Version every time the game starts. Delete the object if you do not want it.
The focus frame	any UIPilot_Btn_...	Button > Selected Color tints the glowing frame on the focused button, Highlighted Color the fainter frame under the mouse, Pressed Color the frame while clicking. Normal Color is transparent on purpose: a button at rest is just its icon and label.
An icon	the <code>Icon</code> child of a button or row	Image > Source Image. The icons are in <code>Assets/UIPilot/Art/</code> ; any white sprite of your own works.
The colour mood	the <code>Bloom</code> objects under <code>UIPilot_..._Backdrop</code> , and <code>UIPilot_..._Panel</code>	Image > Color on each. The four blooms and the wash behind them make the blue drift; change them and the whole menu changes mood.
The glass panel	UIPilot_..._Band	Image > Color. Keep it nearly opaque: it is the surface the white text is read on.
The decorations	<code>Wave</code> , <code>Frame</code> , <code>Scanlines</code> , <code>Cross</code> objects	Delete any you do not want. Nothing depends on them.
Button label	the <code>Text</code> child of a button	TextMeshPro > Text. Keep the GameObject's <i>name</i> unchanged.

To change	Select	Edit
Where the menu sits	<code>UIPilot_..._Spacer</code>	The title block sits at the top of the glass panel and the buttons at the bottom. Delete the Spacer to bring the buttons up under the title.
How much of the scene shows through	<code>UIPilot_..._Panel</code>	Image > Color (alpha). Lower it and the game behind the menu is clearer; raise it and the menu becomes a solid blue screen.
Font	any text object	TextMeshPro > Font Asset. The themes use open-licence fonts from <code>Assets/UIPilot/Art/Fonts</code> (Michroma, Young Serif, Crimson Pro, Work Sans, Silkscreen, Tektur) that you may ship in your game; see <code>Third-Party Notices.txt</code> .

Your changes are safe. **Build UI never rebuilds a panel that already has all of its buttons**, so clicking it again does not undo your styling.

5.6 Keep the object names

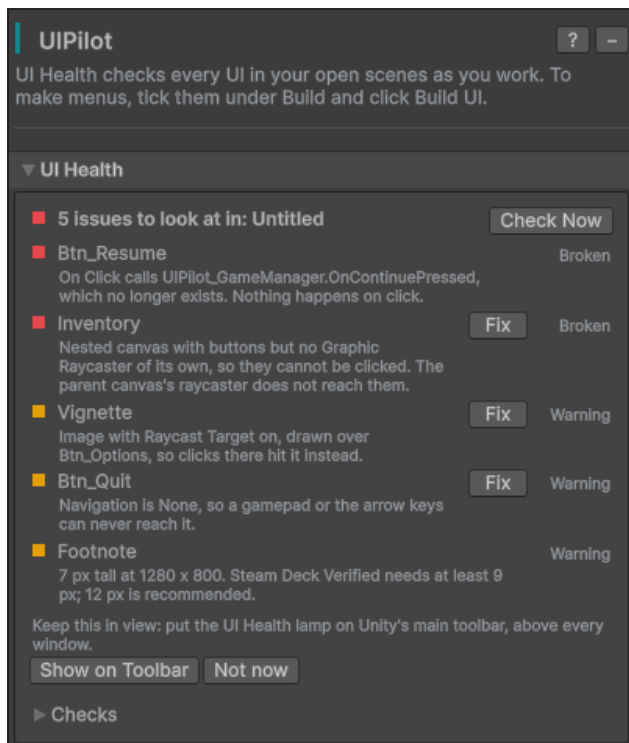
UIPilot recognises its objects by name. If you rename `UIPilot_Canvas`, a panel, a button or `UIPilot_GameManager`, UIPilot treats it as missing. Rename the visible *text* as much as you like, but leave the GameObject names as they are.

6. UI Health: catch broken UI as you work

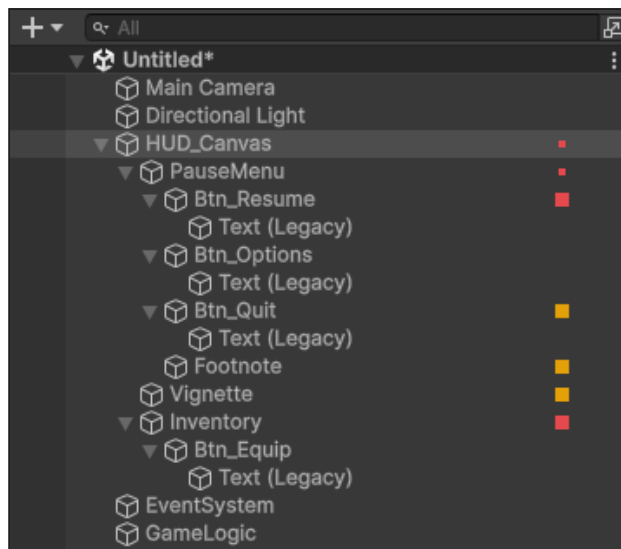
UI Health checks every UI in your open scenes, including UI that UIPilot did not build, for the mistakes that make a control silently fail. It runs by itself: a moment after you stop editing, when you save or open a scene, and just before Play mode starts. When you build, it checks every scene the build includes, open or not. Checking never changes anything. A change is made only when you click **Fix**.

Where you see it

Where	What it shows
Tools > UIPilot, UI Health section	One row per issue: what is wrong, where, and what it means for the player. Click the name to select the object. Fix makes the change the row describes, as one step you can undo with Ctrl+Z . A row without Fix needs a decision from you.
Hierarchy	A lamp at the right of each object with an issue (red: <i>Broken</i> , amber: <i>Warning</i>), and a small lamp on the objects above it, so a folded Hierarchy still shows where to look. Hover a lamp to read the issue.
Main toolbar	A lamp and a count, visible above every window. Click it to open the UI Health section. Unity hides toolbar items that packages add, so the first time, click Show on Toolbar in the UI Health section. Later, Lamp on Unity's main toolbar under Checks turns it on and off.
Console	One warning when you enter Play mode, and one per scene when you build, only when there is something to look at. The build always continues.



UI Health on a test scene with one problem of each kind. Click a name to select it; **Fix** is one undoable step.



The Hierarchy: a lamp on each object with an issue, and a small one on the objects above it.



The main toolbar, once switched on: the worst lamp and the number of issues.

What it checks

Check	What it finds	What Fix does
Click events	A button, toggle, slider, dropdown or input field whose event (On Click, On Value Changed, ...) calls an object that was deleted, a method that was renamed or removed, or has no function selected. Nothing happens when the player clicks.	No Fix: select the control and pick the method again in the Inspector.
EventSystem	No EventSystem in the open scenes, more than one, one without an input module, or <i>StandaloneInputModule</i> in a project whose Active Input Handling is <i>Input System Package (New)</i> , which throws an error every frame.	Adds an EventSystem, adds the input module the project needs, or swaps in <i>InputModuleUIInputModule</i> .
Graphic Raycasters	A Canvas with buttons but no Graphic Raycaster, so nothing on it can be clicked. A nested Canvas needs its own: its parent's raycaster does not reach it.	Adds a Graphic Raycaster.
Blocked buttons	An image or text with Raycast Target on, drawn over the middle of a button, which takes the button's clicks. Typical culprits: a full-screen fade left at alpha 0, a vignette, a label that overlaps a button.	Turns off Raycast Target on the object in the way.
Gamepad navigation	A control with Navigation set to <i>None</i> (a gamepad or the arrow keys can never reach it), or <i>Explicit</i> with no neighbours set (once reached, the player is stuck on it).	Sets Navigation to <i>Automatic</i> .

Check	What it finds	What Fix does
Text size (Steam Deck)	Text shorter than 9 px at 1280 x 800, Valve's minimum for Steam Deck Verified (12 px is recommended), worked out from each Canvas Scaler. Hidden panels are included.	No Fix: click the row to select every text it lists, then raise the font size.

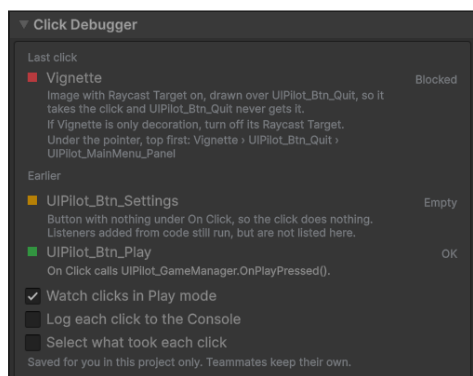
Why it does not cry wolf

A check that keeps raising false alarms gets ignored, so UI Health only reports what it is sure of:

- Menus you show one at a time (main, pause, options) are usually all switched on in the Editor, stacked on top of each other. A panel with buttons of its own is treated as another screen, never as a blocker. Only something in the way that has no buttons of its own is reported.
- Draw order is compared only where it is certain: inside one canvas, or an overlay canvas with a higher Sort Order over a lower one.
- An event entry switched **Off** in the Inspector is left alone.
- Any raycaster counts, including a custom one.
- In a build, the EventSystem check is skipped: another scene or a prefab often adds one at runtime.

If a check does not suit your project, open **Checks** at the bottom of the UI Health section and switch it off. The toolbar lamp and the two Console warnings can be switched off there too. These settings are saved for you in this project only; teammates keep their own.

7. Click Debugger: what took that click?



Three clicks on a generated main menu. Play runs its method. Settings has nothing under On Click. Quit is covered by a HUD vignette with Raycast Target on, so the vignette takes the click: the row names both, and lists what was under the pointer.

When a button does nothing in Play mode, the reason is usually invisible: an image drawn over it, a switched-off CanvasGroup, a missing raycaster, an empty On Click. The Click Debugger shows it. Press **Play**, click anywhere in the Game view, and the **Click Debugger** section of **Tools > UIPilot** says what took the click and why:

You see	It means
OK, On Click calls GameManager.OnPlayPressed()	The click reached a control, and this is what it runs.
Empty	A Button took the click, but nothing is set under On Click (or every entry is broken or switched Off).

You see	It means
Off	The control has Interactable off, or sits inside a CanvasGroup that does. The row names which.
Blocked	Something with Raycast Target on is drawn over a control and takes its clicks. The row names both.
Broken	A control under the pointer cannot receive clicks at all: its Raycast Target is off, its canvas has no Graphic Raycaster, a CanvasGroup has Blocks Raycasts off, or a mask hides it there. Also shown when there is no EventSystem or input module.
—	Nothing clickable was there: a background, or empty space.

The last click is shown in full, with the objects under the pointer from top to bottom; the five before it get a line each. Click a name to select that object. The clicks from your last Play session stay listed after you stop, until you press Play again.

Three switches sit under the list. **Watch clicks in Play mode** is on by default. **Log each click to the Console** writes one line per click; clicking the line selects the object, and it works with the window closed. **Select what took each click** selects it as you play. Like UI Health's, these are saved for you in this project only.

It works with any UI in the scene, not only UIPilot's, and with the Input System package or the old Input Manager. It only reads: nothing in your scene is changed, and nothing of it goes into your build.

8. The UIPilot window

UI Health

The first section. Everything in it is described in section 6.

Click Debugger

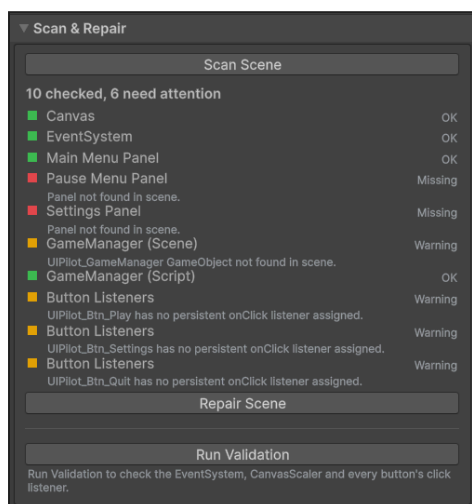
The second section. Everything in it is described in section 7.

Build

Control	What it does
Menu checkboxes	Choose which menus Build UI creates.
Theme	The look Build UI generates (section 5.4). Empty means the built-in Soft Club look. Your choice is remembered.
Restore Default Themes	Only shown when one of the seven default themes is missing from the project. Recreates the missing ones, exactly as shipped. Themes that still exist are never changed.
Build UI	Creates the canvas, the ticked menus, an EventSystem if the scene has none, the <code>UIPilot_GameManager</code> script and GameObject, and connects every button. Anything that already exists and is complete is left alone. The button is greyed out while Unity is compiling a build in progress, and the result appears in a status row beneath it.

Control	What it does
Apply Theme to Existing Menus	Restyles the menus already in the scene with the selected theme. Layout, names and wiring are untouched. Undo is supported.
Quick Clear	After you confirm, removes <code>UIPilot_Canvas</code> and the <code>UIPilot_GameManager</code> GameObject from the scene. Your <code>UIPilot_GameManager.cs</code> file is not deleted.

Scan & Repair



Scan & Repair after scanning a scene that has only a main menu and no GameManager yet. Each row shows a lamp and its status in words. **Repair Scene** appears because some rows are not OK.

Scan Scene checks eight things and shows one row for each. Every row has a status lamp and the status written out beside it (*OK*, *Warning*, *Missing*, *Broken*), so you never have to rely on the colour alone.

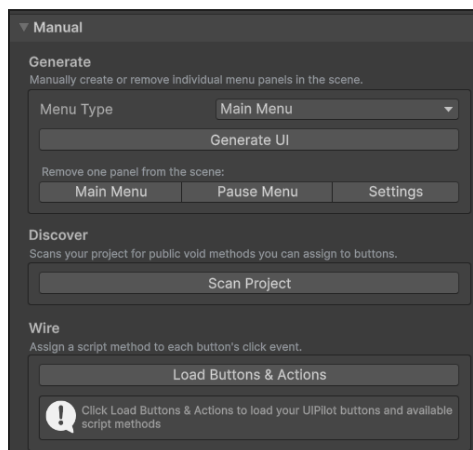
Row	It checks that
Canvas	<code>UIPilot_Canvas</code> exists and its CanvasScaler is set to Scale With Screen Size, 1920 x 1080.
EventSystem	The scene has an EventSystem.
Main Menu / Pause Menu / Settings Panel	The panel exists and still has all of its buttons.
GameManager (Scene)	The <code>UIPilot_GameManager</code> GameObject exists and has its component.
GameManager (Script)	<code>UIPilot_GameManager.cs</code> exists in the project.
Button Listeners	Every UIPilot button has a click listener saved in the scene.

If any row is not *OK*, a **Repair Scene** button appears. It fixes every listed problem in one step (recreates what is missing, re-attaches the component, reconnects the buttons) and then lists each repair as *Fixed* or *Failed*. One **Ctrl+Z** undoes the whole repair.

Scan Scene reports a panel you chose not to build as missing, and Repair Scene would create it. If you do not want that panel, ignore the row.

Run Validation is a second, quicker check of the EventSystem, the CanvasScaler and each button's listener. Rows that can be fixed automatically show a **Fix** button.

Manual



Manual: the three steps Build UI performs for you, available one at a time.

Use these only when you want to do one step by hand. Build UI does all of them for you.

Part	What it does
Generate	Generate UI creates the menu panel chosen in <i>Menu Type</i> . The row under <i>Remove one panel from the scene</i> removes the Main Menu, Pause Menu or Settings panel.
Discover	Scan Project lists every method a button can call: <code>public</code> , returns <code>void</code> , takes no parameters, and belongs to a MonoBehaviour script in your project.
Wire	Load Buttons & Actions , pick a method for each button, then Apply Bindings . Buttons that share a name (for example <i>Settings</i> on both the Main and Pause menus) appear as one row, and your choice is applied to all of them. The script you pick must be on a GameObject in the scene.

9. What UIPilot changes, and how your work is protected

- **It only edits its own objects.** UIPilot creates and modifies GameObjects whose names start with `UIPilot_`. Two exceptions, both deliberate: it adds an `EventSystem` if your scene has none, and it removes any *other* Canvas that contains `UIPilot_` objects (left over from older UIPilot versions).
- **UI Health reads everything and changes nothing on its own.** It looks at every UI in your open scenes, but it changes an object that is not UIPilot's own only when you click **Fix** on that object's row. Each Fix is one step that **Ctrl+Z** undoes.
- **Your script is protected.** UIPilot rewrites `UIPilot_GameManager.cs` only when the script or the `UIPilot_GameManager` GameObject is missing. If you have edited any of the `On...Pressed` methods, it shows a confirmation dialog first, and **Cancel** leaves your file untouched.
- **Scene changes can be undone** with Ctrl+Z. Writing the script file cannot.
- **The Console is cleared** when you click Build UI, Scan Scene or Repair Scene, and when you confirm Quick Clear, so that UIPilot's messages are easy to read.
- **What ends up in your build:** `UIPilot_GameManager.cs` and only the artwork in `Assets/UIPilot/Art` that your menus use: about 300 KB for Soft Club, about 2 MB for Fantasy RPG with its parchment. All of UIPilot's own code is inside an `Editor` folder, which Unity never includes in builds.

10. Troubleshooting

What you see	Why	What to do
Menu text is invisible, or a <i>TMP Importer</i> window pops up	TMP Essential Resources are not imported.	Window > TextMeshPro > Import TMP Essential Resources. Then Quick Clear and Build UI .
"Waiting for Unity to compile..." and nothing more happens, or the window shows a red <i>Stalled</i> row	Another script in your project has a compile error, so Unity cannot finish compiling.	Fix the errors shown in the Console. The buttons are wired automatically after the next successful compile.
"UIPilot_GameManager type could not be resolved"	The generated script is not in Unity's default assembly.	Keep <code>UIPilot_GameManager.cs</code> directly under <code>Assets/</code> , outside any folder that has an Assembly Definition, then click Build UI again.
Buttons do not react in Play mode	Usually one of the problems UI Health checks for: a wrong input module, a missing raycaster, or something covering the buttons.	Open Tools > UIPilot and look at UI Health. Click Fix on the row it shows. Still stuck? Click the button in Play mode and read the Click Debugger row (section 7).
The UI Health lamp is not on the main toolbar	Unity hides toolbar items that packages add, until they are switched on.	Click Show on Toolbar in the UI Health section, or tick Lamp on Unity's main toolbar under Checks . If neither is there, a later Unity version has changed its toolbar: right-click an empty part of the main toolbar (or click its <code>:</code> menu) and tick UIPilot > UI Health .
UI Health reports something you did on purpose	Every check follows common practice, and some projects differ.	Switch that check off under Checks in the UI Health section.
Clicking Settings only logs a warning	The scene has no Settings panel.	Tick Settings Menu and click Build UI .
The pause menu never appears	Nothing opens it yet. This is by design.	Add the script from section 5.2.
Scan Scene shows a <i>Button Listeners</i> warning	A button lost its connection, for example after the GameManager was deleted.	Click Repair Scene .
After updating UIPilot, the menus still have the old look	Complete panels are never rebuilt, to protect your edits.	Quick Clear , then Build UI .
Scan Scene says an object is missing, but it is in the scene	The object was renamed.	Restore its original <code>UIPilot_...</code> name (see section 5.6).

11. Limitations

- UGUI only. UI Toolkit is not supported.
- One `UIPilot_Canvas` per scene, with up to three menus: Main, Pause, Settings.
- The Settings menu ships with three settings: volume, fullscreen and quality. Others (key bindings, separate music and effects volumes) are yours to add; see section 5.3.
- The menu sizes are tuned for desktop and console screens (1920 x 1080 reference). On phones the buttons come out smaller than a comfortable touch target. For a mobile game, select each button and raise **Layout Element > Preferred Height** to about 130, and **Preferred Width** on the buttons, settings rows, title and rule to about 640.
- Buttons are connected to methods named `On<ButtonName>Pressed`. Buttons you add yourself can be connected in **Manual > Wire** if their name starts with `UIPilot_Btn_`.
- UI Health checks the scenes open in Edit mode, and every scene in a build. It does not see UI that your scripts create or change at runtime, or a prefab open in Prefab Mode. The blocked-button and text-size checks skip World Space canvases.
- The Click Debugger lists the listeners set in the Inspector. Listeners your scripts add with `AddListener` still run, but are not listed. It explains the left mouse button, a pen tip or a touch; other buttons are not watched.
- Tested on Unity 6000.3.11f1 (Windows) with the Built-in Render Pipeline.

12. Support

Made by NomadStudios — NomadStudios47@outlook.com

If something does not behave as this guide describes, send an email with your Unity version and the Console output. Every report gets a reply.

Licensed under the [Unity Asset Store EULA](#).